

F Y E O

Natgold

Repo: <https://github.com/fpco/natgold-presale>

Security Review Update: May 04, 2026

Reviewer: balthasar@gofyeo.com



Security Code Review of the Natgold Presale

New security issues, 1

After the development team implemented the latest updates, FYEO conducted a review of the modifications. The primary goal of this evaluation was to ensure the continued robustness of the program's security features, safeguarding user funds and maintaining the overall integrity of the program.

General Updates:

The update spans both the Solidity smart contract and the Rust backend services. On the contract side, the changes rename `copperWallet` to `custodianWallet` throughout the `NatGoldPresale` contract, add a new `updateCustodianWallet` function gated by the `requireAdmin` modifier with a zero-address check and event emission, and update `transferUsdt` to send accumulated USDT to the renamed `custodianWallet`.

The Solidity changes are safe from a storage layout perspective since the variable occupies the same slot position and type as before, preserving upgrade compatibility for the UUPS proxy. The new `updateCustodianWallet` function is consistent with the existing admin trust model, and the deployment module and test suite were updated accordingly with proper coverage for unauthorized access, zero-address rejection, and event verification. On the Rust backend side, the update introduces Account Abstraction relay support through new `aa/session` and `aa/rpc` endpoints backed by a Pimlico client, with a method allowlist restricting forwarded RPC calls to a defined set of AA operations and sender validation ensuring user operations match the authenticated wallet. The KYC flow received substantial hardening including session key binding for document verification to prevent stale webhook processing, duplicate verification ID deduplication, transactional row locking with `FOR UPDATE` to eliminate race conditions, and a retry mechanism allowing up to three document re-submissions for specific failure reasons such as poor image quality, selfie mismatch, or name mismatch.

The `kyc-token` CLI gained a new `signer-from-aws-key` command for deriving Ethereum addresses from base64-encoded DER public keys, and the previously commented-out KYC approval check on the `pac` endpoint was re-enabled, closing what had been an open bypass. Additional changes include a new `unauthenticated sardine/client-tokens` endpoint for obtaining Sardine client tokens, a `me` endpoint returning current KYC state to authenticated users, and the refactoring of the app constructor into `AppConfig` and `AppDeps` structs. The webhook signature validation was refactored from `is_some_and` to `is_none_or` with equivalent logic, and customer feedback errors after a successful status update are now logged rather than propagated, preventing a Sardine feedback outage from rolling back an already-committed KYC decision.

SARDINE CLIENT-TOKENS IS UNAUTHENTICATED

Finding ID: FYEO-NATP-01

Severity: **Low**

Status: **Remediated**

Description

The `sardine_client_token` endpoint is registered outside the `authenticated_routes` group and accepts an arbitrary `customer_id` from the request body. It forwards this directly to Sardine's `/v1/auth/client-tokens` without verifying wallet ownership or session state.

This is intentionally unauthenticated: the frontend needs a Sardine client token to initialize the SDK before the user has connected their wallet or signed T&C, so no session exists yet. The `referenceId` is generated server-side, preventing correlation or replay.

The practical risk is limited to Sardine API quota abuse through unauthenticated spam.

Proof of Issue

File name: `backend/natgold-presale/src/lib.rs`

Line number: 873

```
.route("/sardine/client-tokens", post(sardine_client_token))
```

Severity and Impact Summary

The endpoint is intentionally public, leaks no sensitive data, and grants no privileged access. The only practical risk is API quota exhaustion.

Recommendation

Add per-IP rate limiting to this endpoint as part of the broader rate-limiting effort.

Commit Hash Reference:

For transparency and reference, the security review was conducted on the specific commit hash for the Natgold repository. The commit hash for the reviewed versions is as follows:

4e485047cc7e22990eea7f7bf93aa6e91fe9d6d5

Remediations were submitted with the commit hash 9ad6dbb66d1455d9055a22b59f9471ef5467e78d.

Conclusion:

In conclusion, the security aspects of the Natgold program remain robust and unaffected by the recent updates. Users can confidently interact with the protocol, assured that their funds are well-protected. The commitment to security exhibited by the development team is commendable, and we appreciate the ongoing efforts to prioritize the safeguarding of user assets.